TypeScript

Základy a třídy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

20.10.2026

CC BY-NC-SA 4.0

Úvod

Co je TypeScript?

TypeScript je

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává **statickou typovou kontrolu**

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává **statickou typovou kontrolu**

- Vytvořen a udržován Microsoftem

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává **statickou typovou kontrolu**

- Vytvořen a udržován Microsoftem
- Open-source projekt

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává **statickou typovou kontrolu**

- Vytvořen a udržován Microsoftem
- Open-source projekt
- Transpiluje se do JavaScriptu

Co je TypeScript?

TypeScript je nadstavba JavaScriptu, která přidává **statickou typovou kontrolu**

- Vytvořen a udržován Microsoftem
- Open-source projekt
- Transpiluje se do JavaScriptu
- Kompatibilní se všemi prostředími, kde běží JavaScript

Co je TypeScript?

TypeScript =

Co je TypeScript?

TypeScript = JavaScript + **typy**

Co je TypeScript?

TypeScript = JavaScript + **typy**

```
// JavaScript
function add(a, b) {
  return a + b;
}
```

Co je TypeScript?

TypeScript = JavaScript + **typy**

```
// JavaScript
function add(a, b) {
  return a + b;
}

// TypeScript
function add(a: number, b: number): number {
  return a + b;
}
```

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu
- Lepší **IntelliSense** a automatické doplňování v editoru

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu
- Lepší **IntelliSense** a automatické doplňování v editoru
- **Refaktoring** kódu je bezpečnější a jednodušší

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu
- Lepší **IntelliSense** a automatické doplňování v editoru
- **Refaktoring** kódu je bezpečnější a jednodušší
- Dokumentace pomocí typů

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu
- Lepší **IntelliSense** a automatické doplňování v editoru
- **Refaktoring** kódu je bezpečnější a jednodušší
- Dokumentace pomocí typů
- Lepší **škálovatelnost** pro velké projekty

Proč TypeScript?

Jaké jsou podle vás výhody použití TypeScriptu?

- Odhalení chyb již při **kompilaci**, ne až za běhu
- Lepší **IntelliSense** a automatické doplňování v editoru
- **Refaktoring** kódu je bezpečnější a jednodušší
- Dokumentace pomocí typů
- Lepší **škálovatelnost** pro velké projekty
- Podpora **moderních JavaScript** funkcí

Proč TypeScript?

Příklad problému, který TypeScript řeší:

```
// JavaScript - runtime error
function getUser(id) {
  return { name: "John", age: 30 };
}

const user = getUser(1);
console.log(user.nmae);
// undefined - typo!
```

Proč TypeScript?

```
// TypeScript - compile-time error
function getUser(id: number): { name: string; age: number } {
  return { name: "John", age: 30 };
}

const user = getUser(1);
console.log(user.nmae);
// Error: Property 'nmae' does not exist on type ...
```

Instalace a nastavení

TypeScript můžete nainstalovat pomocí npm:

```
npm install -g typescript
```

Instalace a nastavení

TypeScript můžete nainstalovat pomocí npm:

```
npm install -g typescript
```

Ověření instalace:

```
tsc --version
```

Instalace a nastavení

Vytvoření TypeScript souboru:

```
echo 'console.log("Hello TypeScript");' > hello.ts
```

Instalace a nastavení

Vytvoření TypeScript souboru:

```
echo 'console.log("Hello TypeScript");' > hello.ts
```

Kompilace do JavaScriptu:

```
tsc hello.ts
```

Instalace a nastavení

Vytvoření TypeScript souboru:

```
echo 'console.log("Hello TypeScript");' > hello.ts
```

Kompilace do JavaScriptu:

```
tsc hello.ts
```

Výsledkem je soubor `hello.js`, který lze spustit:

```
node hello.js
```

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Vytvoří se soubor `tsconfig.json` s konfigurací kompilátoru

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Vytvoří se soubor `tsconfig.json` s konfigurací kompilátoru

Základní možnosti konfigurace:

- `target` - verze JavaScriptu (ES5, ES6, …)

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Vytvoří se soubor `tsconfig.json` s konfigurací kompilátoru

Základní možnosti konfigurace:

- `target` - verze JavaScriptu (ES5, ES6, ...)
- `module` - systém modulů (CommonJS, ES6, ...)

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Vytvoří se soubor `tsconfig.json` s konfigurací kompilátoru

Základní možnosti konfigurace:

- `target` - verze JavaScriptu (ES5, ES6, ...)
- `module` - systém modulů (CommonJS, ES6, ...)
- `strict` - přísná kontrola typů

Instalace a nastavení

Inicializace TypeScript projektu:

```
tsc --init
```

Vytvoří se soubor `tsconfig.json` s konfigurací kompilátoru

Základní možnosti konfigurace:

- `target` - verze JavaScriptu (ES5, ES6, ...)
- `module` - systém modulů (CommonJS, ES6, ...)
- `strict` - přísná kontrola typů
- `outDir` - adresář pro zkompilované soubory

Základy TypeScriptu

Typový systém

TypeScript používá **statický typový systém**

Typový systém

TypeScript používá **statický typový systém**

Co to znamená?

Typový systém

TypeScript používá **statický typový systém**

Co to znamená?

- Typy proměnných jsou známy **v době kompilace**

Typový systém

TypeScript používá **statický typový systém**

Co to znamená?

- Typy proměnných jsou známy **v době kompilace**
- Kompilátor kontroluje, zda používáte správné typy

Typový systém

TypeScript používá **statický typový systém**

Co to znamená?

- Typy proměnných jsou známy **v době kompilace**
- Kompilátor kontroluje, zda používáte správné typy
- Chyby jsou odhaleny **před spuštěním** programu

Typový systém

TypeScript má dva způsoby určení typu:

Typový systém

TypeScript má dva způsoby určení typu:

1. **Explicitní anotace typů**

Typový systém

TypeScript má dva způsoby určení typu:

1. **Explicitní anotace typů**

```
let age: number = 30;
let name: string = "John";
```

Typový systém

TypeScript má dva způsoby určení typu:

1. **Explicitní anotace typů**

```
let age: number = 30;
let name: string = "John";
```

2. **Inference typů** (odvození)

Typový systém

TypeScript má dva způsoby určení typu:

1. **Explicitní anotace typů**

```
let age: number = 30;
let name: string = "John";
```

2. **Inference typů** (odvození)

```
let age = 30;       // TypeScript infers: number
let name = "John"; // TypeScript infers: string
```

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)
- `string` - řetězce

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)
- `string` - řetězce
- `boolean` - logické hodnoty

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)
- `string` - řetězce
- `boolean` - logické hodnoty
- `null` a `undefined`

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)
- `string` - řetězce
- `boolean` - logické hodnoty
- `null` a `undefined`
- `object` - objekty

Základní typy

TypeScript obsahuje stejné základní typy jako JavaScript:

- `number` - čísla (celá i desetinná)
- `string` - řetězce
- `boolean` - logické hodnoty
- `null` a `undefined`
- `object` - objekty
- A přidává nové: `any`, `unknown`, `never`, `void`

Základní typy

Number

```
let decimal: number = 6;
let hex: number = 0xf00d;
let binary: number = 0b1010;
let octal: number = 0o744;
let big: number = 1_000_000;
```

Základní typy

String

```
let color: string = "blue";
let fullName: string = `John Doe`;
let greeting: string = `Hello, ${fullName}`;
```

Základní typy

String

```
let color: string = "blue";
let fullName: string = `John Doe`;
let greeting: string = `Hello, ${fullName}`;
```

Boolean

```
let isDone: boolean = false;
let isActive: boolean = true;
```

Základní typy

Array

```
// Two ways to write array types
let list: number[] = [1, 2, 3];
let list2: Array<number> = [1, 2, 3];

let names: string[] = ["John", "Jane"];
```

Základní typy

Tuple - pole s pevným počtem prvků různých typů

```
let tuple: [string, number];
tuple = ["hello", 10]; // OK
tuple = [10, "hello"]; // Error

// Accessing tuple elements
console.log(tuple[0]); // "hello"
console.log(tuple[1]); // 10
```

Základní typy

Enum - pojmenované konstanty

```
enum Color {
  Red,
  Green,
  Blue
}

let c: Color = Color.Green;
console.log(c); // 1 (index)
```

Základní typy

Enum lze použít i s vlastními hodnotami

```
// With custom values
enum Status {
  Active = "ACTIVE",
  Inactive = "INACTIVE"
}
```

Základní typy

Any - vypnutí typové kontroly

```
let notSure: any = 4;
notSure = "maybe a string";
notSure = false; // OK

// Use sparingly!
```

Základní typy

Unknown - bezpečnější alternativa k any

```
let value: unknown = 4;
value = "string"; // OK

// But you must check type before using
if (typeof value === "string") {
  console.log(value.toUpperCase());
}
```

Základní typy

Void - absence návratové hodnoty

```
function logMessage(message: string): void {
  console.log(message);
  // no return statement
}
```

Základní typy

Never - funkce nikdy nevrací hodnotu

```
function error(message: string): never {
  throw new Error(message);
}

function infiniteLoop(): never {
  while (true) {}
}
```

Funkce a typy funkcí

Funkce v TypeScriptu můžeme typovat několika způsoby:

Funkce a typy funkcí

Funkce v TypeScriptu můžeme typovat několika způsoby:

```
// Named function with types
function add(a: number, b: number): number {
  return a + b;
}
```

Funkce a typy funkcí

Funkce v TypeScriptu můžeme typovat několika způsoby:

```
// Named function with types
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function with types
const subtract = (a: number, b: number): number => {
  return a - b;
};
```

Funkce a typy funkcí

Volitelné parametry

```
function greet(name: string, greeting?: string): string {
  if (greeting) {
    return `${greeting}, ${name}`;
  }
  return `Hello, ${name}`;
}

greet("John");                  // "Hello, John"
greet("John", "Good day");  // "Good day, John"
```

Funkce a typy funkcí

Otazník za parametrem označuje, že parametr je volitelný a jedná se o „syntax sugar“ pro ekvivalentní definici například

```
string | undefined
```

Funkce a typy funkcí

Výchozí hodnoty parametrů

```
function greet(
  name: string,
  greeting: string = "Hello"
): string {
  return `${greeting}, ${name}`;
}

greet("John");              // "Hello, John"
greet("John", "Hi");        // "Hi, John"
```

Funkce a typy funkcí

Rest parametry

```
function sum(...numbers: number[]): number {
  return numbers.reduce((total, n) => total + n, 0);
}

sum(1, 2, 3);        // 6
sum(1, 2, 3, 4, 5); // 15
```

Funkce a typy funkcí

Typ funkce

```
// Function type
let myAdd: (a: number, b: number) => number;

myAdd = function(a: number, b: number): number {
  return a + b;
};
```

Funkce a typy funkcí

```
function calculate(
  a: number,
  b: number,
  operation: (x: number, y: number) => number
): number {
  return operation(a, b);
}

calculate(2, 3, (x, y) => x + y); // 5
calculate(2, 3, (x, y) => x * y); // 6
```

Union a Intersection typy

Union typy umožňují proměnné mít více povolených typů

Union a Intersection typy

Union typy umožňují proměnné mít více povolených typů

```
let value: string | number;

value = "hello";  // OK
value = 42;       // OK
value = true;     // Error: boolean is not assignable
```

Union a Intersection typy

Použití union typů s funkcemi:

```
function printId(id: string | number) {
  console.log(`ID: ${id}`);
}

printId(101);      // OK
printId("202");    // OK
```

Union a Intersection typy

Intersection typy kombinují více typů do jednoho

```
type Person = {
  name: string;
  age: number;
};
type Employee = {
  employeeId: number;
  department: string;
};
type Staff = Person & Employee;
```

Union a Intersection typy

```
const john: Staff = {
  name: "John",
  age: 30,
  employeeId: 1234,
  department: "IT"
};
```

Type Guards - zúžení typu

Type guards jsou

Type Guards - zúžení typu

Type guards jsou techniky pro zúžení (narrowing) typu proměnné v určitém kontextu

Type Guards - zúžení typu

Type guards jsou techniky pro zúžení (narrowing) typu proměnné v určitém kontextu

TypeScript umí automaticky inferovat konkrétnější typ na základě kontextu

Type Guards - zúžení typu

typeof - pro primitivní typy

```
function processValue(value: string | number) {
  if (typeof value === "string") {
    // TypeScript knows that value is string
    return value.toUpperCase();
  } else {
    // TypeScript knows that value is number
    return value.toFixed(2);
  }
}
```

Type Guards - zúžení typu

instanceof - pro třídy a objekty

```
class Dog { bark() { console.log("Woof!"); } }
class Cat { meow() { console.log("Meow!"); } }

function makeSound(animal: Dog | Cat) {
  if (animal instanceof Dog) {
    animal.bark(); // TypeScript knows it's a Dog
  } else {
    animal.meow(); // TypeScript knows it's a Cat
  }
}
```

Type Guards - zúžení typu

in operator - kontrola existence vlastnosti

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function move(animal: Fish | Bird) {
  if ("swim" in animal) {
    animal.swim(); // TypeScript knows it's a Fish
  } else {
    animal.fly();  // TypeScript knows it's a Bird
  }
}
```

Type Guards - zúžení typu

Custom type guard - vlastní funkce pro kontrolu typu

```
interface Cat { meow: () => void; }
interface Dog { bark: () => void; }

function isCat(animal: Cat | Dog): animal is Cat {
  return (animal as Cat).meow !== undefined;
}
```

Type Guards - zúžení typu

```
function makeSound(animal: Cat | Dog) {
  if (isCat(animal)) {
    animal.meow(); // TypeScript knows it's a Cat
  } else {
    animal.bark(); // TypeScript knows it's a Dog
  }
}
```

Type Guards - zúžení typu

```
function makeSound(animal: Cat | Dog) {
  if (isCat(animal)) {
    animal.meow(); // TypeScript knows it's a Cat
  } else {
    animal.bark(); // TypeScript knows it's a Dog
  }
}
```

Klíčové slovo `is` v návratovém typu říká TypeScriptu, že funkce je type guard

Discriminated Unions (Tagged Unions)

Discriminated Union je

Discriminated Unions (Tagged Unions)

Discriminated Union je pattern, kde každá varianta union typu má

Discriminated Unions (Tagged Unions)

Discriminated Union je pattern, kde každá varianta union typu má společnou vlastnost (**diskriminátor**)

Discriminated Unions (Tagged Unions)

Discriminated Union je pattern, kde každá varianta union typu má společnou vlastnost (**diskriminátor**)

Velmi užitečné pro reprezentaci stavů nebo variant

Discriminated Unions (Tagged Unions)

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; size: number }
  | { kind: "rectangle"; width: number; height: number };
```

Discriminated Unions (Tagged Unions)

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; size: number }
  | { kind: "rectangle"; width: number; height: number };
```

Vlastnost `kind` je **diskriminátor** - jednoznačně určuje typ

Discriminated Unions (Tagged Unions)

```
function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.size ** 2;
    case "rectangle":
      return shape.width * shape.height;
  }
}
```

Discriminated Unions (Tagged Unions)

```
function getArea(shape: Shape): number {
  switch (shape.kind) {
    case "circle":
      return Math.PI * shape.radius ** 2;
    case "square":
      return shape.size ** 2;
    case "rectangle":
      return shape.width * shape.height;
  }
}
```

TypeScript v každé větvi `switch` ví přesný typ!

Discriminated Unions (Tagged Unions)

Praktický příklad - API response:

```
type ApiResponse<T> =
  | { status: "success"; data: T }
  | { status: "error"; error: string }
  | { status: "loading" };
```

Discriminated Unions (Tagged Unions)

```
function handleResponse(response: ApiResponse<User>) {
  switch (response.status) {
    case "success":
      console.log(response.data.name); break; // OK
    case "error":
      console.error(response.error); break; // OK
    case "loading":
      console.log("Loading..."); break; // OK
  }
}
```

Discriminated Unions (Tagged Unions)

Výhody Discriminated Unions:

Discriminated Unions (Tagged Unions)

Výhody Discriminated Unions:

- **Exhaustive checking** - TypeScript kontroluje, že jste ošetřili všechny případy

Discriminated Unions (Tagged Unions)

Výhody Discriminated Unions:

- **Exhaustive checking** - TypeScript kontroluje, že jste ošetřili všechny případy
- **Type safety** - přesná typová kontrola v každé větvi

Discriminated Unions (Tagged Unions)

Výhody Discriminated Unions:

- **Exhaustive checking** - TypeScript kontroluje, že jste ošetřili všechny případy
- **Type safety** - přesná typová kontrola v každé větvi
- **Čitelnost** - explicitní reprezentace stavů

Discriminated Unions (Tagged Unions)

Výhody Discriminated Unions:

- **Exhaustive checking** - TypeScript kontroluje, že jste ošetřili všechny případy
- **Type safety** - přesná typová kontrola v každé větvi
- **Čitelnost** - explicitní reprezentace stavů
- Skvělé pro **state management**

Type aliases a Type assertions

Type alias vytvoří nové jméno pro typ

Type aliases a Type assertions

Type alias vytvoří nové jméno pro typ

```
type Point = {
  x: number;
  y: number;
};

const point: Point = { x: 10, y: 20 };
```

Type aliases a Type assertions

```
// Type alias for union
type ID = string | number;
function printId(id: ID) {
  console.log(id);
}
```

Type aliases a Type assertions

```
// Type alias for union
type ID = string | number;
function printId(id: ID) {
  console.log(id);
}

// Type alias for function
type GreetFunction = (name: string) => string;
const greet: GreetFunction = (name) => {
  return `Hello, ${name}`;
};
```

Type aliases a Type assertions

Type assertion řekne kompilátoru, že znáte typ lépe

Type aliases a Type assertions

Type assertion řekne kompilátoru, že znáte typ lépe

```
// Two syntaxes
let someValue: unknown = "this is a string";

let strLength1: number = (someValue as string).length;
let strLength2: number = (<string>someValue).length;
```

Type aliases a Type assertions

Type assertion řekne kompilátoru, že znáte typ lépe

```
// Two syntaxes
let someValue: unknown = "this is a string";

let strLength1: number = (someValue as string).length;
let strLength2: number = (<string>someValue).length;
```

Pozor! Type assertion **neprovádí konverzi**, pouze říká kompilátoru „věř mi, vím co dělám“

Type aliases a Type assertions

```
// Practical example with DOM
const input =
  document.getElementById("myInput") as HTMLInputElement;
input.value = "Hello";
```

Type aliases a Type assertions

```
// Practical example with DOM
const input =
  document.getElementById("myInput") as HTMLInputElement;
input.value = "Hello";
```

Bez type assertion by TypeScript nevěděl, že element má vlastnost `value`

Objektově orientované programování

Základy OOP

Objektově orientované programování (OOP) je

Základy OOP

Objektově orientované programování (OOP) je programovací paradigma založené na konceptu **objektů**

Základy OOP

Objektově orientované programování (OOP) je programovací paradigma založené na konceptu **objektů**

Objekty obsahují:

- **Data** (vlastnosti, fieldy, atributy)

Základy OOP

Objektově orientované programování (OOP) je programovací paradigma založené na konceptu **objektů**

Objekty obsahují:

- **Data** (vlastnosti, fieldy, atributy)
- **Chování** (metody, funkce)

Základy OOP

Základní principy OOP:

Základy OOP

Základní principy OOP:

1. **Zapouzdření** (Encapsulation)

Základy OOP

Základní principy OOP:

1. **Zapouzdření** (Encapsulation)
2. **Dědičnost** (Inheritance)

Základy OOP

Základní principy OOP:

1. **Zapouzdření** (Encapsulation)
2. **Dědičnost** (Inheritance)
3. **Polymorfismus** (Polymorphism)

Základy OOP

Základní principy OOP:

1. **Zapouzdření** (Encapsulation)
2. **Dědičnost** (Inheritance)
3. **Polymorfismus** (Polymorphism)
4. **Abstrakce** (Abstraction)

Základy OOP

Proč používat OOP?

Základy OOP

Proč používat OOP?

- Lepší **organizace** kódu

Základy OOP

Proč používat OOP?

- Lepší **organizace** kódu
- **Znovupoužitelnost** kódu

Základy OOP

Proč používat OOP?

- Lepší **organizace** kódu
- **Znovupoužitelnost** kódu
- Snadnější **údržba** a rozšiřování

Základy OOP

Proč používat OOP?

- Lepší **organizace** kódu
- **Znovupoužitelnost** kódu
- Snadnější **údržba** a rozšiřování
- Modelování **reálného světa**

Základy OOP

Proč používat OOP?

- Lepší **organizace** kódu
- **Znovupoužitelnost** kódu
- Snadnější **údržba** a rozšiřování
- Modelování **reálného světa**
- Lépe škálovatelné pro **velké projekty**

Třídy v TypeScriptu

Třída je

Třídy v TypeScriptu

Třída je šablona (blueprint) pro vytváření objektů

Třídy v TypeScriptu

```
class Person {
  name: string;
  age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  greet(): string { return `Hello, I am ${this.name}`; }
}
```

Třídy v TypeScriptu

Vytvoření instance třídy:

```
const john = new Person("John", 30);
console.log(john.greet()); // "Hello, I am John"
console.log(john.age);     // 30
```

Třídy v TypeScriptu

Rozdíl oproti JavaScript ES6 třídám?

Třídy v TypeScriptu

Rozdíl oproti JavaScript ES6 třídám?

TypeScript přidává:

- **Typovou kontrolu** vlastností a parametrů

Třídy v TypeScriptu

Rozdíl oproti JavaScript ES6 třídám?

TypeScript přidává:

- **Typovou kontrolu** vlastností a parametrů
- **Modifikátory přístupu** (public, private, protected)

Třídy v TypeScriptu

Rozdíl oproti JavaScript ES6 třídám?

TypeScript přidává:

- **Typovou kontrolu** vlastností a parametrů
- **Modifikátory přístupu** (public, private, protected)
- **Abstraktní třídy** a metody

Třídy v TypeScriptu

Rozdíl oproti JavaScript ES6 třídám?

TypeScript přidává:

- **Typovou kontrolu** vlastností a parametrů
- **Modifikátory přístupu** (public, private, protected)
- **Abstraktní třídy** a metody
- Lepší podporu pro **rozhraní**

Konstruktory a vlastnosti

Konstruktor je speciální metoda volaná při vytvoření instance

Konstruktory a vlastnosti

```
class Rectangle {
  width: number;
  height: number;

  constructor(width: number, height: number) {
    this.width = width;
    this.height = height;
  }

  area(): number { return this.width * this.height; }
}
```

Konstruktory a vlastnosti

TypeScript nabízí **zkrácenou syntaxi** pro vlastnosti:

```
// Long way
class Rectangle {
  width: number;
  height: number;

  constructor(width: number, height: number) {
    this.width = width;
    this.height = height;
  }
}
```

Konstruktory a vlastnosti

```
// Short way (parameter properties)
class Rectangle {
  constructor(
    public width: number,
    public height: number
  ) {}
}
```

Konstruktory a vlastnosti

```
// Short way (parameter properties)
class Rectangle {
  constructor(
    public width: number,
    public height: number
  ) {}
}
```

Obě verze dělají totéž, ale zkrácená syntaxe je čitelnější!

Konstruktory a vlastnosti

Výchozí hodnoty vlastností:

```
class Counter {
  count: number = 0;

  increment(): void {
    this.count++;
  }
}

const counter = new Counter();
console.log(counter.count); // 0
```

Statické vlastnosti a metody

Statické členy patří

Statické vlastnosti a metody

Statické členy patří třídě samotné, ne instancím

Statické vlastnosti a metody

Statické členy patří třídě samotné, ne instancím

Označují se klíčovým slovem `static`

Statické vlastnosti a metody

Statické členy patří třídě samotné, ne instancím

Označují se klíčovým slovem `static`

Přistupuje se k nim přes název třídy, ne přes instanci

Statické vlastnosti a metody

```
class MathHelper {
  static PI = 3.14159;

  static circleArea(radius: number): number {
    return this.PI * radius ** 2;
  }

  static circleCircumference(radius: number): number {
    return 2 * this.PI * radius;
  }
}
```

Statické vlastnosti a metody

Použití statických členů:

```
// Access through class name
console.log(MathHelper.PI);                 // 3.14159
console.log(MathHelper.circleArea(5));      // 78.53975

// CANNOT access through instance
const helper = new MathHelper();
console.log(helper.PI);  // Error: Property 'PI' is static
```

Statické vlastnosti a metody

Kdy používat statické členy?

Statické vlastnosti a metody

Kdy používat statické členy?

- **Utility funkce** související s třídou

Statické vlastnosti a metody

Kdy používat statické členy?

- **Utility funkce** související s třídou
- **Konstanty** sdílené všemi instancemi

Statické vlastnosti a metody

Kdy používat statické členy?

- **Utility funkce** související s třídou
- **Konstanty** sdílené všemi instancemi
- **Factory metody** pro vytváření instancí

Statické vlastnosti a metody

Kdy používat statické členy?

- **Utility funkce** související s třídou
- **Konstanty** sdílené všemi instancemi
- **Factory metody** pro vytváření instancí
- **Počítadla** instancí

Factory pomocí statické metody

```
class User {
  constructor(
    public name: string,
    public email: string
  ) {}

  static fromJSON(json: string): User {
    const data = JSON.parse(json);
    return new User(data.name, data.email);
  }
}
```

Factory pomocí statické metody

```
const jsonString =
  '{"name": "John", "email": "john@example.com"}';
const user = User.fromJSON(jsonString);
console.log(user.name);  // "John"
console.log(user.email); // "john@example.com"
```

Počítadlo instancí

```
class Database {
  private static instanceCount = 0;

  constructor(public name: string) {
    Database.instanceCount++;
  }

  static getInstanceCount(): number {
    return Database.instanceCount;
  }
}
```

Počítadlo instancí

```
const db1 = new Database("users");
const db2 = new Database("products");

console.log(Database.getInstanceCount()); // 2
```

Zapouzdření

Modifikátory přístupu

Zapouzdření znamená

Modifikátory přístupu

Zapouzdření znamená skrytí interních detailů objektu a poskytnutí veřejného rozhraní

Modifikátory přístupu

Zapouzdření znamená skrytí interních detailů objektu a poskytnutí veřejného rozhraní

TypeScript nabízí tři modifikátory přístupu:

Modifikátory přístupu

Zapouzdření znamená skrytí interních detailů objektu a poskytnutí veřejného rozhraní

TypeScript nabízí tři modifikátory přístupu:

- `public` - přístupné odkudkoliv (výchozí)

Modifikátory přístupu

Zapouzdření znamená skrytí interních detailů objektu a poskytnutí veřejného rozhraní

TypeScript nabízí tři modifikátory přístupu:

- `public` - přístupné odkudkoliv (výchozí)
- `private` - přístupné pouze uvnitř třídy

Modifikátory přístupu

Zapouzdření znamená skrytí interních detailů objektu a poskytnutí veřejného rozhraní

TypeScript nabízí tři modifikátory přístupu:

- `public` - přístupné odkudkoliv (výchozí)
- `private` - přístupné pouze uvnitř třídy
- `protected` - přístupné uvnitř třídy a jejích potomků

Modifikátory přístupu

Public (výchozí)

```
class Person {
  public name: string; // explicitly public
  age: number;         // implicitly public

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}
```

Modifikátory přístupu

```
const person = new Person("John", 30);
console.log(person.name); // OK
console.log(person.age);  // OK
```

Modifikátory přístupu

Private

```
class BankAccount {
  private balance: number = 0;

  deposit(amount: number): void {
    this.balance += amount;
  }
  getBalance(): number {
    return this.balance;
  }
}
```

Modifikátory přístupu

```
const account = new BankAccount();
account.deposit(100);
console.log(account.getBalance()); // 100
console.log(account.balance);
// Error: Property 'balance' is private
```

Modifikátory přístupu

```
const account = new BankAccount();
account.deposit(100);
console.log(account.getBalance()); // 100
console.log(account.balance);
// Error: Property 'balance' is private
```

Private vlastnosti jsou přístupné pouze uvnitř třídy!

Modifikátory přístupu

Protected

```
class Animal {
  protected name: string;

  constructor(name: string) {
    this.name = name;
  }
}
```

Modifikátory přístupu

```
class Dog extends Animal {
  bark(): void {
    console.log(`${this.name} says woof!`); // OK - protected
  }
}
```

Modifikátory přístupu

```
const dog = new Dog("Rex");
dog.bark(); // "Rex says woof!"
console.log(dog.name);
// Error: Property 'name' is protected
```

Gettery a settery

Gettery a **settery** poskytují kontrolovaný přístup k vlastnostem

Gettery a settery

Gettery a **settery** poskytují kontrolovaný přístup k vlastnostem

```
class Person {
  private _age: number = 0;

  get age(): number {
    return this._age;
  }
```

Gettery a settery

```
  set age(value: number) {
    if (value < 0) {
      throw new Error("Age cannot be negative");
    }
    this._age = value;
  }
}
```

Gettery a settery

Použití getterů a setterů:

```
const person = new Person();
person.age = 30;           // calls setter
console.log(person.age); // calls getter, output: 30

person.age = -5;           // Error: Age cannot be negative
```

Gettery a settery

Použití getterů a setterů:

```
const person = new Person();
person.age = 30;           // calls setter
console.log(person.age); // calls getter, output: 30

person.age = -5;           // Error: Age cannot be negative
```

Přístup vypadá jako k běžné vlastnosti, ale máme kontrolu!

Gettery a settery

Praktický příklad - validace emailu:

```
class User {
  private _email: string = "";

  get email(): string {
    return this._email;
  }
```

Gettery a settery

```
  set email(value: string) {
    if (!value.includes("@")) {
      throw new Error("Invalid email");
    }
    this._email = value;
  }
}
```

Readonly vlastnosti

Readonly vlastnosti lze nastavit pouze

Readonly vlastnosti

Readonly vlastnosti lze nastavit pouze v konstruktoru

Readonly vlastnosti

Readonly vlastnosti lze nastavit pouze v konstruktoru

```
class Person {
  readonly id: number;
  readonly birthDate: Date;

  constructor(id: number, birthDate: Date) {
    this.id = id;
    this.birthDate = birthDate;
  }
}
```

Readonly vlastnosti

```
const person = new Person(1, new Date("1990-01-01"));
console.log(person.id); // 1

person.id = 2;
// Error: Cannot assign to 'id' because it is a read-only
property
```

Readonly vlastnosti

```
const person = new Person(1, new Date("1990-01-01"));
console.log(person.id); // 1

person.id = 2;
// Error: Cannot assign to 'id' because it is a read-only
property
```

Užitečné pro vlastnosti, které by se neměly měnit po vytvoření objektu!

Readonly vlastnosti

Readonly s parameter properties:

```
class Person {
  constructor(
    public readonly id: number,
    public name: string
  ) {}
}

const person = new Person(1, "John");
person.name = "Jane"; // OK
person.id = 2;        // Error: readonly
```

Dědičnost

Extends klíčové slovo

Dědičnost umožňuje

Extends klíčové slovo

Dědičnost umožňuje vytvořit novou třídu na základě existující

Extends klíčové slovo

Dědičnost umožňuje vytvořit novou třídu na základě existující

Nová třída (**potomek**) zdědí vlastnosti a metody z **rodičovské třídy**

Extends klíčové slovo

Dědičnost umožňuje vytvořit novou třídu na základě existující

Nová třída (**potomek**) zdědí vlastnosti a metody z **rodičovské třídy**

Používá se klíčové slovo `extends`

Extends klíčové slovo

```
class Animal {
  constructor(public name: string) {}

  move(distance: number): void {
    console.log(`${this.name} moved ${distance}m`);
  }
}
```

Extends klíčové slovo

```
class Dog extends Animal {
  bark(): void {
    console.log(`${this.name} says woof!`);
  }
}
```

Extends klíčové slovo

```
const dog = new Dog("Rex");
dog.bark();             // "Rex says woof!"
dog.move(10);           // "Rex moved 10m" - inherited method
console.log(dog.name); // "Rex" - inherited property
```

Extends klíčové slovo

Výhody dědičnosti:

Extends klíčové slovo

Výhody dědičnosti:

- **Znovupoužití** kódu z rodičovské třídy

Extends klíčové slovo

Výhody dědičnosti:

- **Znovupoužití** kódu z rodičovské třídy
- **Rozšíření** funkcionality rodičovské třídy

Extends klíčové slovo

Výhody dědičnosti:

- **Znovupoužití** kódu z rodičovské třídy
- **Rozšíření** funkcionality rodičovské třídy
- Vytvoření **hierarchie** tříd

Extends klíčové slovo

Výhody dědičnosti:

- **Znovupoužití** kódu z rodičovské třídy
- **Rozšíření** funkcionality rodičovské třídy
- Vytvoření **hierarchie** tříd
- **DRY princip** (Don’t Repeat Yourself)

Super a přístup k rodičovské třídě

Klíčové slovo `super` umožňuje

Super a přístup k rodičovské třídě

Klíčové slovo `super` umožňuje přístup k rodičovské třídě

Super a přístup k rodičovské třídě

Klíčové slovo `super` umožňuje přístup k rodičovské třídě

Používá se:

- V konstruktoru pro volání rodičovského konstruktoru

Super a přístup k rodičovské třídě

Klíčové slovo `super` umožňuje přístup k rodičovské třídě

Používá se:

- V konstruktoru pro volání rodičovského konstruktoru
- Pro volání rodičovských metod

Super a přístup k rodičovské třídě

Super v konstruktoru

```
class Animal {
  constructor(public name: string) {}
}

class Dog extends Animal {
  constructor(name: string, public breed: string) {
    super(name); // calls parent constructor
    // must be first in constructor!
  }
}
```

Super a přístup k rodičovské třídě

```
const dog = new Dog("Rex", "German Shepherd");
console.log(dog.name);  // "Rex"
console.log(dog.breed); // "German Shepherd"
```

Super a přístup k rodičovské třídě

```
const dog = new Dog("Rex", "German Shepherd");
console.log(dog.name);  // "Rex"
console.log(dog.breed); // "German Shepherd"
```

Bez volání `super()` by došlo k chybě!

Super pro volání metod

```
class Animal {
  move(distance: number): void {
    console.log(`Animal moved ${distance}m`);
  }
}
class Dog extends Animal {
  move(distance: number): void {
    super.move(distance); // calls parent method
    console.log("with enthusiasm!");
  }
}
```

Super pro volání metod

```
const dog = new Dog("Rex");
dog.move(10);
// "Animal moved 10m"
// "with enthusiasm!"
```

Přepisování metod

Přepisování metod (method overriding) znamená

Přepisování metod

Přepisování metod (method overriding) znamená definovat metodu v potomkovi se stejným jménem jako v rodiči

Přepisování metod

Přepisování metod (method overriding) znamená definovat metodu v potomkovi se stejným jménem jako v rodiči

Potomek pak používá svou vlastní implementaci

Přepisování metod

```
class Animal {
  makeSound(): void {
    console.log("Some generic animal sound");
  }
}

class Dog extends Animal {
  makeSound(): void {
    console.log("Woof!");
  }
}
```

```
class Cat extends Animal {
  makeSound(): void {
    console.log("Meow!");
  }
}
```

Přepisování metod

```
const dog = new Dog();
const cat = new Cat();

dog.makeSound(); // "Woof!"
cat.makeSound(); // "Meow!"
```

Přepisování metod

```
const dog = new Dog();
const cat = new Cat();

dog.makeSound(); // "Woof!"
cat.makeSound(); // "Meow!"
```

Každá třída má svou vlastní implementaci!

Přepisování metod

TypeScript 4.3+ umožňuje explicitně označit přepisování pomocí klíčového slova `override`:

```
class Dog extends Animal {
  override makeSound(): void {
    console.log("Woof!");
  }
}
```

Přepisování metod

TypeScript 4.3+ umožňuje explicitně označit přepisování pomocí klíčového slova `override`:

```
class Dog extends Animal {
  override makeSound(): void {
    console.log("Woof!");
  }
}
```

Kompilátor pak kontroluje, že metoda v rodiči existuje. Pomáhá při refaktoringu a prevenci chyb

Závěr

Shrnutí

Co jsme se naučili:

Shrnutí

Co jsme se naučili:

TypeScript základy

- Typový systém, základní typy
- Union, Intersection typy
- Type Guards a Discriminated Unions
- Type aliases a assertions

Shrnutí

Objektově orientované programování

- Třídy, konstruktory, vlastnosti
- Statické členy
- Zapouzdření (public, private, protected)
- Dědičnost (extends, super)

Shrnutí

Dědičností jsme ale s OOP neskončili

Shrnutí

Dědičností jsme ale s OOP neskončili

Příště nás čeká **polymorfismus**, **rozhraní** a **SOLID principy**